



PROJECT-BASED LEARNING

PHASE-I EVALUATION

NEXUS

AI-Powered Distributed Data Processing
& Compute Orchestration Platform

Team ID: OS-DBMS-V-2026-T099

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026-27



Team & Mentor Details

Project Information

Team ID: OS-DBMS-V-2026-T099
Team Name: Avengers
Semester: 5th
Domain: AI / DBMS / OS / Dist. Sys.
Project Type: Full-Stack Web Application

Project Title

NEXUS: AI-Powered Distributed Data
Processing & Compute Orchestration Platform

Team Members

1. Team Lead

Pareek, Raghvander – 240211322

2. Team Member

Kumar Singh, Tanishk – 240211960

3. Team Member

Kumar Singh, Tushar – 240211961

4. Team Member

Raj Kathait, Soyam – 240211683

Mentor Details

Mentor: Mr.Yogesh Lohumi

Interactions: 2

Problem Statement & Motivation

Problem Statement

Modern organisations generate large amounts of data, but analysing this data can be difficult for users lacking SQL knowledge. Analytical queries require variable CPU, memory, and runtime. Traditional systems lack an integrated mechanism connecting **query understanding, optimisation, scheduling, and resource management**.

NEXUS proposes a unified platform combining AI data interaction with DBMS, OS, and Distributed Systems principles.

Why is this a problem?

- Users need SQL knowledge for analysis.
- Complex queries consume heavy resources.
- Jobs have varying execution requirements.
- Query & resource management are siloed.

NEXUS Approach

- Natural-language interface for queries.
- AI converts user queries to SQL.
- SQL is validated, optimized & scheduled.
- Concurrent execution with visual output.

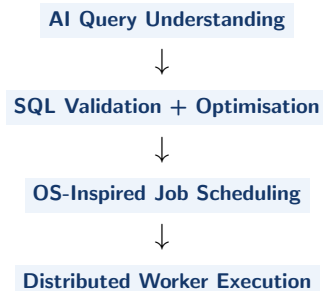
State of the Art & Research Gap

Existing Solutions

- Database Management Systems / SQL Clients
- BI Dashboards & Data-analysis software
- AI Natural Language-to-SQL tools
- Distributed data-processing platforms

Identified Gap AI tools focus heavily on NL-to-SQL generation without execution management. Distributed execution systems target experienced developers.

NEXUS Research Direction



Contribution: Integration of NL-to-SQL, query optimisation, and OS scheduling into one platform.

Project Objectives

1. Interactive web app for data analysis.
2. Upload and manage structured datasets.
3. Ask analytical queries in natural language.
4. Convert NL requests to SQL via LLMs.
5. Validate SQL for safety and correctness.
6. Optimise queries before execution.
7. Implement job queue & CPU scheduling.
8. Execute via multiple worker processes.
9. Monitor CPU, memory & worker status.
10. Maintain execution logs and history.
11. Provide visual results & AI explanations.

Key Objective: Transform natural-language analytical requests into validated, optimized, and scheduled computational jobs.

Project Goals & Milestones



Major Project Goals

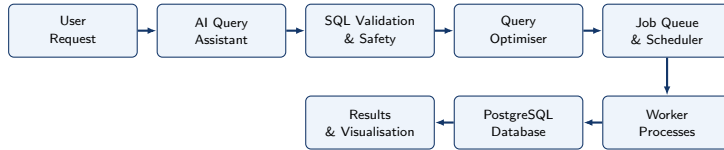
- Role-based user authentication
- Schema understanding & upload engine
- Natural-language query parsing
- SQL safety validation & query optimisation
- Job queue, multi-worker execution & monitoring

Development Milestones

1. Requirement analysis & architecture
2. PostgreSQL database implementation
3. Frontend user interface setup
4. AI NL-to-SQL integration
5. Scheduler & worker engine
6. System testing & performance evaluation

Development Strategy: Build and validate incrementally, verifying each architectural layer prior to complete system-level integration.

Proposed Solution



User Workflow

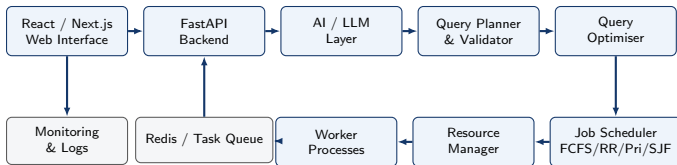
1. Upload/select target dataset
2. Ask query in natural language
3. System validates & optimises generated SQL
4. Job queued, scheduled, and executed
5. View visual results & explanations

Operational Flow

Execution Pipeline

Prompt: “Find top ten cities by revenue” → AI Conversion → Safety Check → Optimiser → OS Scheduler → Multi-Worker Execution → Result Display

System Architecture



Application Layer

- Next.js Frontend & FastAPI Web Server
- JWT Auth & Dataset Management APIs

Systems & Orchestration Layer

- Rule-based Optimisation & Priority Scheduling
- Worker pool execution backed by PostgreSQL



Integration of PBL Course Concepts

Course	Concepts Applied	NEXUS Module
Data Structures in C	Queues, priority queues, hashing, tree/graph structures	Job Scheduler / Query Engine
OOPs with C++	Classes, encapsulation, inheritance, STL paradigms	Query Planner / Workers
Operating Systems	Multi-threading, CPU scheduling (SJF/RR), concurrency	Scheduler / Resource Manager
DBMS	Relational schemas, SQL indexing, execution plans	PostgreSQL / Query Meta-data

System Integration Model

DBMS executes and stores dataset schemas and query state. **Operating Systems** principles handle job queuing, priority dispatching, and process synchronization. **AI** serves as the natural language translation bridge.

Core Technical Integration

Operating Systems

- Process management
- CPU scheduling (FCFS, RR, Priority, SJF)
- Resource allocation
- Worker synchronization

DBMS

- Relational schema design
- SQL & Normalisation
- Indexing & Transactions
- Query execution plans

Artificial Intelligence

- Schema-aware prompts
- Natural Language to SQL
- Self-correcting syntax
- Result summarization

NEXUS Pipeline: NL Query → AI Translation → Validation → Optimisation → OS Scheduling → Worker Execution → Visual Output

Technology Stack & Methodology

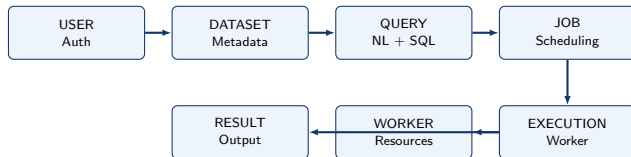
Technology Stack

- ▶ **Frontend:** React / Next.js
- ▶ **Backend:** Python FastAPI
- ▶ **Database:** PostgreSQL
- ▶ **Queue/Scheduler:** Redis / Custom OS Scheduler
- ▶ **AI Orchestration:** LLM API / Local Model
- ▶ **Deployment:** Docker Containers

Development Steps

1. Requirements & Schema Design
2. API & Frontend Skeleton
3. AI Prompting & Validation Engine
4. Scheduler & Worker Pool Engine
5. Integration, Benchmarking & Testing

Key Performance Metrics AI SQL Generation Accuracy (%) — Query Latency (ms) — Scheduler Waiting Time — CPU/Worker Utilization



Core Entities

- Users, Datasets & Schema Mapping
- Generated SQL, Jobs & Execution Metrics

Detailed ER Diagrams will be finalized during Phase II database architecture execution.

Database Responsibilities

- Transactional isolation & Referential integrity
- Persistent execution logging & indexing

Initial Progress & Team Contribution



Phase-I Progress

- Problem definition & scope established
- System architecture & tech stack finalized
- Mapping of OS algorithms to job queues
- Initial PostgreSQL entity definitions

Team Allocation

Member	Role	Responsibilities
Raghvander	Team Lead	Architecture, requirements analysis & module integration
Tanishk	Backend	FastAPI development, REST endpoints & API routes
Tushar Soyam	DB / Query AI / Systems	PostgreSQL schema, query parser & validation logic LLM prompts, OS scheduling algorithm & worker logic

Assumptions & Initial Scope

Assumptions

- ▶ Dataset input format consists of structured CSV files.
- ▶ Primary database operations rely on PostgreSQL.
- ▶ LLM API access remains available for query synthesis.
- ▶ Workers operate in a bounded multi-process environment.

Scope Boundaries

- ▶ Web dashboard for query entry & visualization.
- ▶ Natural-language-to-SQL translation layer.
- ▶ SQL security filter preventing destructive statements.
- ▶ OS-inspired CPU job scheduling algorithms.

Safety Protocol: AI only assists query translation; query execution is strictly controlled by validation checks and OS scheduler rules.

Application Deliverables

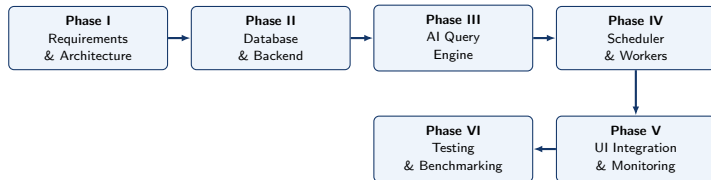
1. Web portal for interactive data analysis
2. Role-based authentication system
3. Dataset parser & automatic schema detector
4. Natural-language query translation module
5. SQL validation and safety guardrails

Systems Deliverables

1. Multi-algorithm job scheduler
2. Concurrent worker execution pool
3. Real-time resource usage monitoring
4. Query history & execution log system
5. Comparative performance benchmark suite

The final software platform integrates **AI, DBMS, Operating Systems, and Distributed Systems** into a single functional analytical application.

Development Roadmap



Phase-I Focus (Current)

- Domain definition, research gap analysis
- Architectural pipeline & database design

Subsequent Focus

- Core backend logic, scheduling engine
- System evaluation & benchmarking

References

1. Sikha Bagui, Marta Malagutti, Riccardo Morelli, and Michael Tamascelli, "Large Language Models in Database Management System Optimization: A Survey," *ACM Transactions on Intelligent Systems and Technology*, Vol. 17, No. 5, 2026. DOI: 10.1145/3829078
2. P. A. Prakash et al., "AI-Powered Transformer Framework for Intelligent Natural Language to SQL Query Generation," in *Proc. Int. Conf. Smart Futuristic Technology (ICSFT)*, 2026. DOI: 10.1109/ICSFT66733.2026.11507944

Supporting Technical Documentation

- PostgreSQL, FastAPI, and Redis Official Technical Manuals
- Operating Systems CPU Scheduling and Concurrency Literature



NEXUS

AI-Powered Distributed Data Processing
& Compute Orchestration Platform

Thank You

Questions & Suggestions

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun